

APPS **Build** Design Launch

On this page



What you'll learn

Requirements

Step 1: Create a web pixel extension

Step 2: Define your web pixel settings

Step 3: Subscribe to events

Step 4: Update your app access scopes

Step 5: Activate a web pixel extension

Step 6: Test the web pixel settings

Step 7: Deploy your extension

Next steps

Build web pixels

Pixels are JavaScript code snippets that run on the online store and collect [behavioral data](#) for marketing campaign optimization and analytics. You can create a web pixel extension to subscribe your app to events that are emitted by Shopify.

What you'll learn

In this tutorial, you'll learn how to do the following tasks:

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

- Deploy your web pixel extension to Shopify.

Requirements

- You're a [user with app development permissions](#) and have created [a dev store](#).
- You've [created an app](#) using Shopify CLI. If you previously installed Shopify CLI, then make sure you're using the [latest version](#).
- Your app can make [authenticated requests](#) to the GraphQL Admin API.
- Your app has the `write_pixels` and `read_customer_events` [access scopes](#). Learn how to [configure your access scopes using Shopify CLI](#).

Step 1: Create a web pixel extension

To create a web pixel extension, you can use Shopify CLI to [generate](#) a starter extension.

1. Navigate to your app directory:

```
Terminal
```

```
$ cd <directory>
```

2. Run the following command to create a new web pixel extension:

```
Terminal
```

```
$ shopify app generate extension --template web_pixel --name my-web-pixel
```

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

```
$ ? What would you like to work in?
$
$ > (1) JavaScript
$    (2) TypeScript
```

After the generation process has completed, a new folder is created in your app directory named `extensions`. This folder contains all of the code for your web pixel extension.

Step 2: Define your web pixel settings

Your web pixel needs to be configured for each store so it can properly track and associate data. The settings for your web pixel can be found at `extensions/<your_extension_name>/shopify.extension.toml`.

Inside of `shopify.extension.toml`, you can define fields that can be used by your web pixel.

In addition, configure privacy settings to integrate your pixel with [cookie banner and data sale opt out](#) features used to help comply with privacy regulations.

To integrate with the cookie banner, use the [latest version of Shopify CLI](#) and configure the purposes required to run your pixel:

1. `analytics`: Your pixel is used to collect information about how customers interact with the site.
2. `marketing`: Your pixel is used to provide ads and marketing communications based on customer interests.
3. `preferences`: Your pixel is used to persist customer preferences and personalize the customer experience.

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

4. `sale_of_data`: Your pixel is used for data sale / sharing.

The following example shows the `shopify.extension.toml` file with a description of each property, as well as a privacy settings configuration that indicates the pixel required `analytics`, `marketing` and `sale_of_data` permissions to run:

 `shopify.extension.toml` 

```
1 type = "web_pixel_extension"
2 name = "my-web-pixel"
3 uid = "5dd0f111-62d8-91e2-8f50-8436afb95d0bfe12168a"
4 runtime_context = "strict"
5
6 [customer_privacy]
7 analytics = true
8 marketing = true
9 preferences = false
10 sale_of_data = "enabled"
11
12 [settings]
13 type = "object"
14
15 # [settings.fields.name]
16 # name = "name"
17 # description = "description"
18 # type = "single_line_text_field"
19 # validations = [{...}]
20
21 # [settings.fields.name]
22 # name = "name"
23 # description = "description"
24 # type = "single_line_text_field"
25 # validations = [{...}]
```

Property	Description
<code>type</code> required	The type of extension. For web pixel extensions, this value is always <code>web_pixel_extension</code> .
<code>name</code>	The name of the web pixel app extension.

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

Property	Description
<code>runtime_context</code> required	The context that the web pixel extension is running in. This value must be set to <code>strict</code> .
<code>customer_privacy</code>	Configure privacy settings definition. If not provided, then it defaults to <code>marketing: true</code> , <code>analytics: true</code> , and <code>sale_of_data: "enabled"</code> . All four privacy purposes need to be explicitly declared if you want to override the default.
<code>customer_privacy.analytics</code> required	A <code>true</code> or <code>false</code> declaration that your pixel requires <code>analytics</code> permissions to run.
<code>customer_privacy.marketing</code> required	A <code>true</code> or <code>false</code> declaration that your pixel requires <code>marketing</code> permissions to run.
<code>customer_privacy.preferences</code> required	A <code>true</code> or <code>false</code> declaration that your pixel requires <code>preferences</code> permissions to run.
<code>customer_privacy.sale_of_data</code> required	A string <code>enum</code> declaration that your pixel requires <code>sale_of_data</code> permissions to run. - <code>enabled</code>: Pixel qualifies as data sale and won't fire if customers opt out of data sale/sharing - <code>disabled</code>: Pixel doesn't qualify as data sale/sharing and will fire when customers opt out of data sale/sharing - <code>ldu</code>: Pixel qualifies as data sale / sharing, but implements a limited data use setting so that it will fire when customers opt out of data sale / sharing.
<code>settings</code> required	Denotes the data type of the settings definition. A <code>type</code> property with the value <code>object</code> must be provided for this section.
<code>settings.fields</code>	Denotes the acceptable metafields for your settings

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

`accountID` and privacy settings configuration of `analytics`, `marketing` and `sale_of_data`, as shown in the following example:

 `shopify.extension.toml` 

```
1 type = "web_pixel_extension"
2 name = "my-web-pixel"
3 uid = "5dd0f111-62d8-91e2-8f50-8436afb95d0bfe12168a"
4 runtime_context = "strict"
5
6 [customer_privacy]
7 analytics = true
8 marketing = true
9 preferences = false
10 sale_of_data = "enabled"
11
12 [settings]
13 type = "object"
14
15 [settings.fields.accountID]
16 name = "Account ID"
17 description = "Account ID"
18 type = "single_line_text_field"
19 validations = [
20   { name = "min", value = "1" }
21 ]
```

Each of the fields that you define within your `settings` has a name, description, and type. The only type that is supported for web pixel is `single_line_text_field`. Validations can be provided as an optional parameter and can be used to define any constraints on a field's value. In the above example, there is a single validation to ensure that the value provided is at minimum length of 1. Further information on validation options can be found on the [metafields validation page](#).

With the settings defined, you can now set up subscriptions to events that are emitted.

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

other developers. You can define a specific event to subscribe to or you can subscribe to all events using `analytics.subscribe`.

You can add an event subscription to your app using the `extensions/<your_extension_name>/src/index.js` file. When you run the [generate extension command](#) to create the extension, a set of basic sample code is created in this file.

You can add code to enable subscription to any target events. For example, you can add the following code that subscribes to all events:

```
extensions/<your_extension_name>/src/index.js

1 import {register} from '@shopify/web-pixels-extension';
2
3 register(async ({analytics, browser, settings}) => {
4   // Retrieve or set your tracking cookies
5   const uid = await browser.cookie.get('your_visitor_cookie');
6   const pixelEndpoint = `https://example.com/pixel?id=${settings.accountID}`;
7
8   // Subscribe to events
9   analytics.subscribe('all_events', (event) => {
10     // Transform the event payload to fit your schema (optional)
11
12     // Send events to third-party servers
13     fetch('https://example.com/pixel', {
14       method: 'POST',
15       body: JSON.stringify(payload),
16       keepalive: true,
17     });
18   });
19 });
```

With this code added, the web pixel is now set to listen to all events that are emitted. Refer to the [Web pixel extension API](#) for additional APIs to support other functions or to subscribe to individual events.

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

[scopes](#) to invoke web pixel mutations in the GraphQL Admin API:

- `write_pixels`: Used to write the web pixel merchant settings
- `read_customer_events`: Used to gain access to customer events

1. In `shopify.app.toml` in the root of your app, add the `write_pixels` and `read_customer_events` access scopes:

```
shopify.app.toml

1 [access_scopes]
2 # Learn more at https://shopify.dev/docs/apps/structure/configuration#access-scopes
3 scopes = "write_pixels,read_customer_events"
```

2. Save your configuration file. If [app dev](#) is running, the scope changes will be applied automatically. If not, start the command to connect the extension to a dev store:

```
Terminal
```

```
$ shopify app dev
```

Note

Connecting the extension to the store is the first step in testing your web pixel extension. However, the web pixel isn't created and configured in the dev store, and isn't subscribed to customer events, until you [run a mutation to create the web pixel](#).

3. In your dev store, go to **Settings > Customer events**.

The name of your app appears in the **App pixels** list with the **Disconnected** status. You'll connect your app and activate the web pixel extension in the [next step](#).

Step 5: Activate a web pixel extension

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

defined, then the mutation will fail.

Note

The simplest way to execute GraphQL queries if you're developing an app is to use the [local GraphQL instance that the CLI provides](#).

1. Execute the following mutation from within your app:

POST https://{shop}.myshopify.com/api/{api_version}/graphql.json

GraphQL mutation

```
1  mutation {
2    # This mutation creates a web pixel, and sets the `accountID` decl
3    webPixelCreate(webPixel: { settings: "{\\"accountID\\":\\"123\\"}" })
4      userErrors {
5        code
6        field
7        message
8      }
9      webPixel {
10       settings
11       id
12     }
13   }
14 }
```

JSON response

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

```
1  {
2    "data": {
3      "webPixelCreate": {
4        "userErrors": [],
5        "webPixel": {
6          "settings": "{\"accountID\":\"123\"}",
7          "id": "gid://shopify/WebPixel/1"
8        }
9      }
10   },
11   "extensions": {
12     "cost": {
13       "requestedQueryCost": 10,
14       "actualQueryCost": 10,
15       "throttleStatus": {
16         "maximumAvailable": 1000,
17         "currentlyAvailable": 990,
18         "restoreRate": 50
19       }
20     }
21   }
22 }
```

2. In your dev store, go to **Settings > Customer events**.

The name of your app should appear in the **App pixels** list with the **Connected** status.

3. Optional. If you need to update your extension code, such as change the settings of `settings.fields.[name]` in `shopify.extension.toml`, and want to see your changes reflected in development, then you need to run the [webPixelUpdate](#) mutation:

```
POST https://{shop}.myshopify.com/api/{api_version}/graphql.json
```

GraphQL mutation



Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

```
1 mutation {
2   webPixelUpdate(id: "gid://shopify/WebPixel/1", webPixel: { setting
3     userErrors {
4       code
5       field
6       message
7     }
8     webPixel {
9       settings
10      id
11    }
12  }
13 }
```

JSON response



```
1 {
2   "data": {
3     "webPixelUpdate": {
4       "userErrors": [],
5       "webPixel": {
6         "settings": "{\"accountID\":\"234\"}",
7         "id": "gid://shopify/WebPixel/1"
8       }
9     }
10  },
11  "extensions": {
12    "cost": {
13      "requestedQueryCost": 10,
14      "actualQueryCost": 10,
15      "throttleStatus": {
16        "maximumAvailable": 1000,
17        "currentlyAvailable": 990,
18        "restoreRate": 50
19      }
20    }
21  }
22 }
```

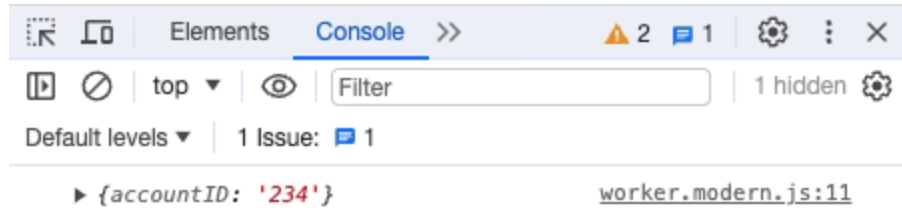
Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

You can test that the web pixel is configured and subscribed to all events emitted on the dev store by completing the following steps:

1. With the server running, open your dev store.
2. Right-click anywhere on the page and select **Inspect**.
3. Click **Console**.

The `accountID` is logged in the console:



Step 7: Deploy your exten

When you're ready to release your changes to users, you can create and release an [app version](#). An app version is a snapshot of your app configuration and all extensions.

1. Navigate to your app directory.
2. Run the following command.

Optionally, you can provide a name or message for the version using the `--version` and `--message` flags.

```
Terminal
$ shopify app deploy
```

Releasing an app version replaces the current active version that's served to stores that

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK

with a `--no-release` flag. You can release the unreleased app version using Shopify CLI's [release](#) command, or through the Dev Dashboard.

Next step

- Consult the [customer events reference](#) to learn about the ways of tracking and sending events with web pixels.
 - Explore the [web pixel extension API](#) to learn about the sandboxing modes for web pixels, and the API references that are available for the sandbox.
-

Was this page helpful? [Yes](#) [No](#)

Updates

[Developer changelog](#)

[Shopify Editions](#)

Business growth

[Shopify Partners Program](#)

[Shopify App Store](#)

[Shopify Academy](#)

Legal

[Terms of service](#)

[API terms of use](#)

Shopify

[About Shopify](#)

[Shopify Plus](#)

Shopify uses cookies to provide necessary site functionality and improve your experience. By using our website, you agree to our [privacy policy](#) and our [cookie policy](#).

OK